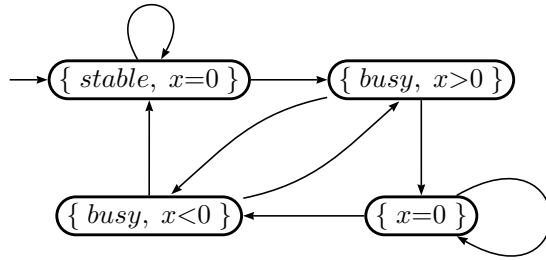


Exercises PVL LTL

Wishnu Prasetya

September 24, 2025

- Below you see a Kripke structure; let's call it M . Give its explicit definition in terms of a tuple etc (see the formal definition in the slides).



- Why don't we have final states there?
- How is the notion of 'execution' defined for a Kripke structure? And what is an 'abstract execution'?
- Give an execution of M that satisfies the property $\mathbf{X} (busy \mathbf{U} (x=0))$. Does M satisfies the property?
- So, given a property Kripke structure M , an (abstract) execution Π , and a property ϕ , and an natural number i , what is the difference between:
 - $M \models \phi$
 - $\Pi \models \phi$
 - $\Pi, i \models \phi$

- Express the following requirements in LTL. Make the necessary assumptions if you have to; but be reasonable.

- P and Q cannot use a resource r simultaneously.

Answer:

$$\Box \neg (use(P, r) \wedge use(Q, r))$$

where $use(P, r)$ is a predicate which is true while and as long as P is using r . Importantly note that it does not represent a program call.

- Whenever P requests access to r , eventually it will get the access.

Answer:

$$\Box (req(P, r) \rightarrow \Diamond use(P, r))$$

where $req(P, r)$ is a predicate which is true while and as long as P is requesting for r .

- (c) Whenever P requests access to r , eventually it will get the access; but only if P persists on maintaining the request.

Answer:

$$\Box(req(P, r) \rightarrow (req(P, r) \text{ U } \neg req(P, r) \vee use(P, r)))$$

- (d) P cannot access r without first requesting it; and it cannot do so (make a request) without first releasing r (if it was busy using r).

Answer:

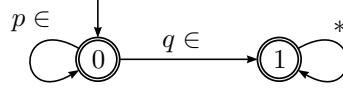
$$\Box((\neg req(P, r) \wedge \neg use(P, r)) \text{ W } (req(P, r) \wedge \neg use(P, r)))$$

$$\Box((use(P, r) \wedge \neg req(P, r)) \text{ W } (\neg use(P, r) \wedge \neg req(P, r)))$$

3. Construct Buchi automata representing the following LTL formulas:

- (a) $p \text{ W } q$, where p, q are atomic propositions.

Answer:



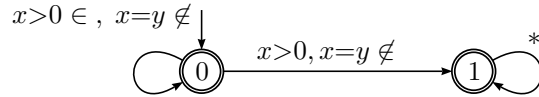
Where the above is a standard Buchi with both states accepting.

- (b) $\neg(x > 0 \text{ U } x = y)$

Answer: Note that $\neg(p \text{ U } q) = (p \wedge \neg q) \text{ W } (\neg p \wedge \neg q)$. So the above property is equivalent to:

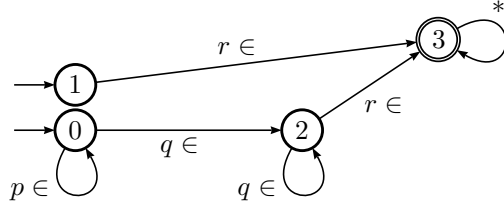
$$(x > 0 \wedge \neg x = y) \text{ W } (\neg x > 0 \wedge \neg x = y)$$

This results in the following standard Buchi automaton. Both states are accepting.



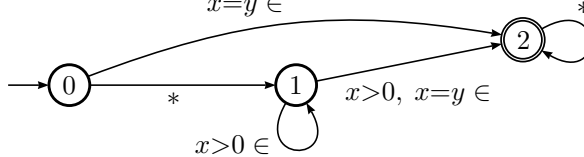
- (c) $p \text{ U } (q \text{ U } r)$, where p, q are atomic propositions.

Answer: The following standard Buchi with $\{0, 1\}$ as initial states, and state 3 as the only accepting state.



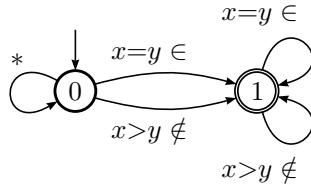
- (d) $(\text{X } x > 0) \text{ U } x = y$

Answer: A standard Buchi, with 2 as the accepting state.



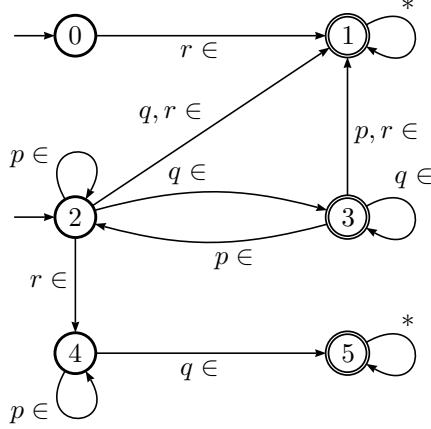
- (e) $\Diamond \Box(x > 0 \rightarrow x = y)$

Answer: Notice first that $x > 0 \rightarrow x = y$ can also be written as $\neg(x > 0) \vee x = y$. Below is a standard Buchi with 1 as the accepting state.



(f) $(p \mathbf{U} q) \mathbf{W} r$

Answer: Using this standard Buchi, with $\{0, 2\}$ as the initial states:



With $F = \{1, 3, 5\}$ as the accepting states. Accepting via 1 describes executions whose prefix repeatedly satisfy $p \mathbf{U} q$, zero or more times, and ends up in q (in the case of at least one time $p \mathbf{U} q$); and then it is followed up with r .

Accepting via 3 describes the scenario of executions that remain in $p \mathbf{U} q$ forever, without ever to go over to r .

Finally, accepting via 5 describes executions whose prefix repeatedly satisfy $p \mathbf{U} q$, zero or more times, and then they go over to r ; thus still owing one future q , but this future q is met (after r).